WNDRVR
SUPPORT NETWORK

VXWORKS 7 SDK VISUAL STUDIO CODE GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 SDK Visual Studio Code Guide, SR0650

10 July 2020

1. ABOUT THE APPLICATION DEVELOPER

The role of the *Application Developer* is write and test software solutions running in a VxWorks Real-time Operating System (RTOS) environment supplied by a software development kit (SDK). VxWorks supports several types of applications, including DKMs, RTPs, and shared libraries.

SDK (Software Development Kit)

A complete system that is ready for application development. It includes the tools necessary to compile, debug, and test software solutions.

RTP (Real-Time Process)

An RTP is an executable application that runs in user-space as a process. An RTP runs in its own environment and cannot directly interact with other resources not mapped on to it. It is a VxWorks application project.

DKM (Downloadable Kernel Module)

A DKM is an application that runs in kernel-mode as tasks, with full access to the system hardware. It is a VxWorks application project.

Shared library

A shared library is a package of software components that reside in user-space and which can be accessed by multiple applications.

2. PREPARING VISUAL STUDIO TO USE A VXWORKS SDK

The application developer can optionally use Visual Studio (VS) with a VxWorks software development kit (SDK).

Before You Begin

- Ensure that you have all the prerequisite information.

Procedure

1. Install the Visual Studio Code (VSCode) IDE for your operating system.
   https://code.visualstudio.com/download
2. Launch VSCode and install the VxWorks SDK extension from the Visual Studio Marketplace.
   https://marketplace.visualstudio.com/items?itemName=WindRiver.VxWorksSDKSupport

   The VxWorks SDK extension supports several project types. You will need to reference your SDK from the extension settings.

3. Browse to the VxWorks SDK Support extension in your VSCode Extension listing.

   Click on the settings "gear icon" and choose **Extension Settings**.

   The extension allows referencing the SDK on a user or workspace basis. Add the top-level location of your SDK (*sdkInstallDir*) to both.

4. Navigate to or create your workspace.

   To create your first workspace:

   1. Select **File > Open Folder** and browse to the location for your VxWorks Application projects.
   2. To make the folder a VSCode Workspace, use **File > Save Workspace As** and give the workspace a name.

   You can now create, build and debug application projects in the workspace you have created. Any projects created while this workspace is active will be placed within the workspace folder. Right clicking on this workspace will give you options to create VxWorks projects in this directory.

3. DEVELOPING APPLICATIONS USING VSCODE

As an application developer, you can use Visual Studio Code (VSCode) to create, build, and debug your applications.

Before You Begin

- Set up VSCode for use with your SDK; see Preparing Visual Studio to Use a VxWorks SDK on page 2.
- If you require QEMU and it is not included in your SDK, install QEMU; see *VxWorks 7 SDK QEMU Installation Guide*.
- Python is used by the helper script provided to start a QEMU virtual target. If you require QEMU and python is not included in your SDK, determine the version of python you need.
  - If the default on your host is version 2*x*, use the python3 command.

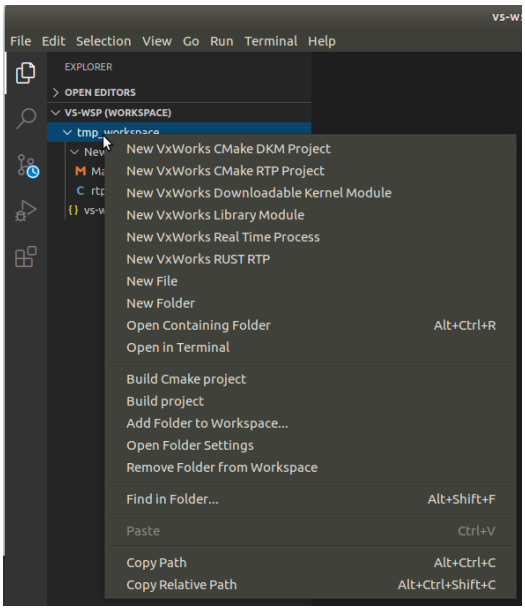
Procedure

1. If required, install or access at least version 3.6 of python.

Host Type	Instructions
Linux	To install directly on the host: `$ sudo apt-get install python3 && python3 -m pip install -U pip`
Windows	To install directly on the host: Download for Windows 10 from the following, ensuring you select **Add Python to PATH**: https://www.python.org/downloads/

2. Create a project with VSCode.

   Right-click in the workspace folder and select the project type (**New VxWorks ...**).



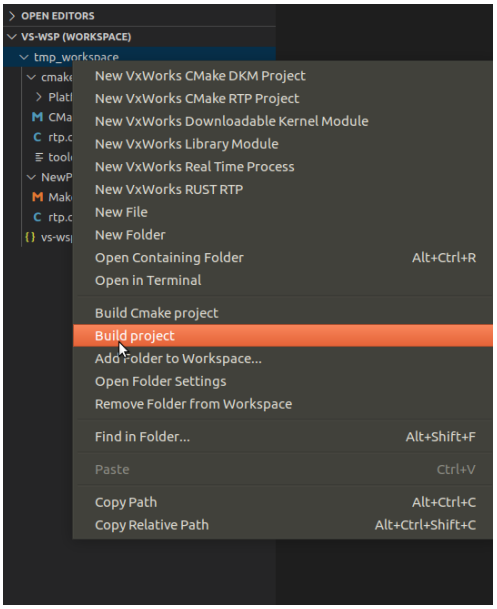
3. In the edit box that appears, enter the name of your project.

   VSCode automatically creates a folder, a top-level C file (with "Hello World" code) and a Makefile for the project you created.

   From this dialog you are able to create C-code projects that use traditional makefiles, CMake tooling, or a Rust-language project (if supported by your VxWorks SDK).

4. Create and edit your source files.
5. Build the project using VSCode.

   Right-click the project name and select **Build project** or **Build Cmake project**.



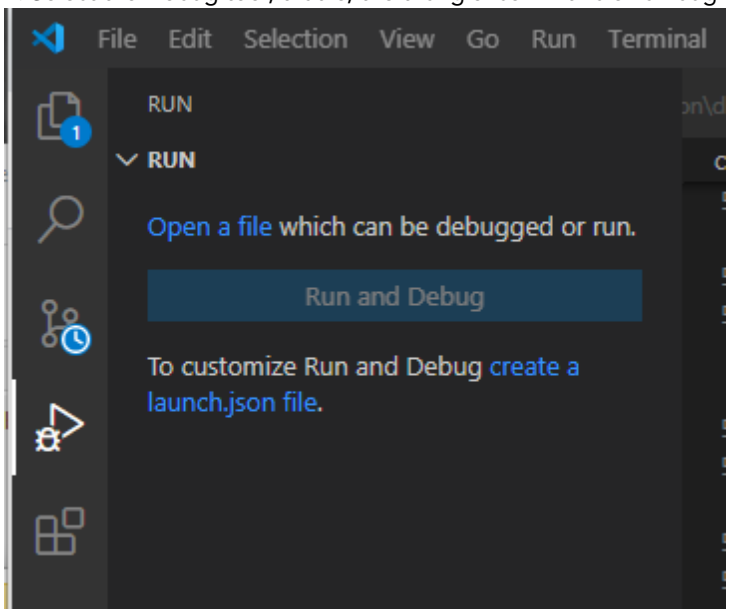
   After your build succeeds, the typical scenario is to load the binary on a running target, debug any issues, iterate as required until you achieve successful execution.

6. Ensure the target is running the kernel image that came with your SDK.

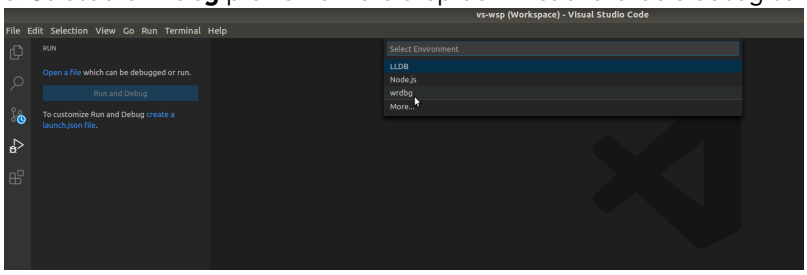
   See the *VxWorks 7 SDK Boot Guide*.

   You can try the **startGenericQemu** setting in the **launch.json** to run a default QEMU-enabled image based on the same architecture as the target SDK VxWorks image. Note that a given target VxWorks image may not work with the generic QEMU.

7. Create a debug configuration (if one does not yet exist).
   1. Select the Debug tool; that is, the triangle icon with a small bug in the lower left corner (see below).



   2. Click **create a launch.json file**.
   3. Select the **wrdbg** profile from the drop-down list of available debug configuration options.



   The **launch.json** configuration file will appear. For example, for an RTP with QEMU, the default file is similar to the image below.

```
.vscode > {} launch.json > ...
1  {
2      // Use IntelliSense to learn about possible attributes.
3      // Hover to view descriptions of existing attributes.
4      // For more information, visit: https://go.microsoft.com/fwlink/?link
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "name": "Run RTP w/ QEMU",
9              "type": "wrdbg",
10             "request": "launch",
11             "target": "<RTP.VXE_FILE>",
12             "cwd": "${workspaceRoot}/<RTP_DIRECTORY>",
13             "valuesFormatting": "parseText",
14             "printCalls": false,
15             "sdkPath": "${config:vscode.vxsdk}",
16             "targetType": "rtp",
17             "startQemu": true,
18             "stopQemu": true,
19             "noDebug": true,
20             "trace": false
21         },
```

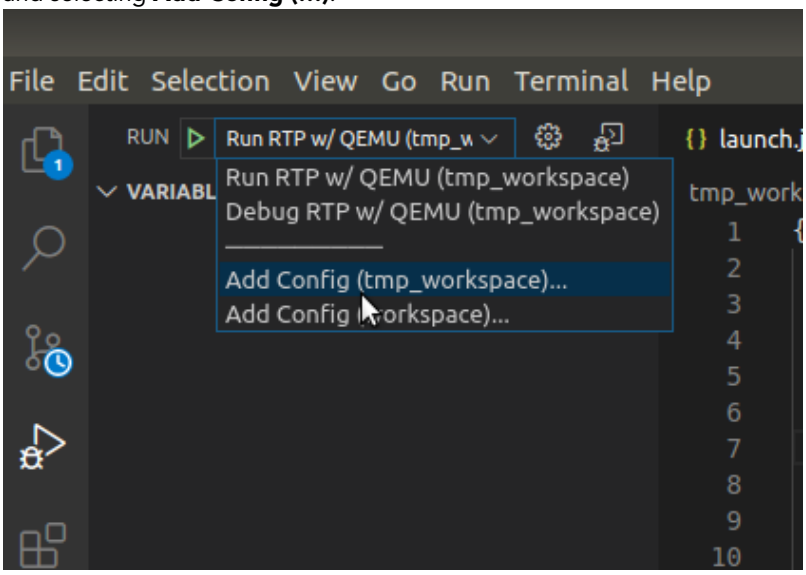
4. To run or debug an RTP, the parameter values that contain placeholders must be updated.

   In the screenshot from the previous step:

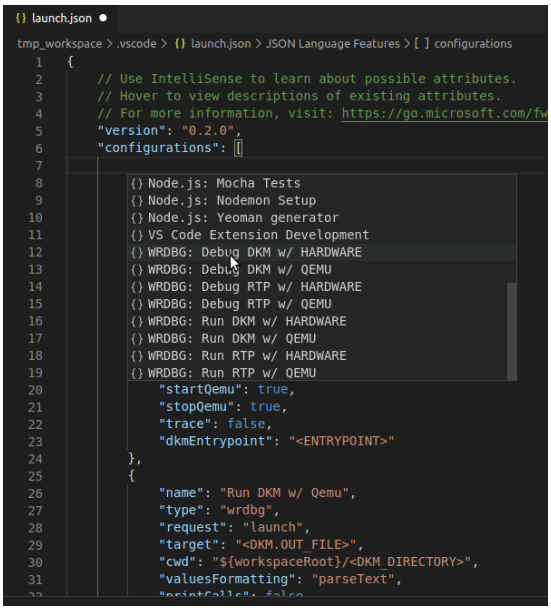
   - Replace the placeholder value of **target** (**<RTP.VXE_FILE>**) with the **.vxe** file name for your application.
   - Replace the placeholder value of **cwd** (**<RTP_DIRECTORY>**) with your project folder name.

   For other options, see VSCode Debug Configuration Settings on page 10.

8. As required, add debugging configurations with different default settings by clicking on the drop-down box in the Debug window and selecting **Add Config (...)**.



The list of options below will appear in your **launch.json** file.



9. Choose one of the options prefixed with **WRDBG:**

   Note that the only difference between the **Run** and **Debug** project configurations is the value set for the **noDebug** parameter in that section.

10. To run/debug an application, press the green side-arrow icon to start the debugging session.

    The VSCode interface provides a range of debugging features allowing you to add breakpoints, step in and out of functions, view symbols, stack traces, process states, and more. Refer to the debugging section of the *VxWorks 7 SDK Visual Studio Code Guide* for more information.

4. DEBUGGING RTP EXAMPLE USING VSCODE

This shows a walkthrough of how an application developer uses Visual Studio Code (VSCode) to debug an RTP application.

Before You Begin

- Follow all the steps to create, build, and set up for debugging; see Developing Applications Using VSCode on page 3.

About This Task

The following debugging features are supported by the VSCode extension in this release. Additional capabilities may be available in future releases, as required.

- Loading symbol files for RTPs, DKMs, and Shared Libraries
- Connecting to the target using a specified IP
- Running the RTP/DKM on target
- Setting multiple breakpoints
- Thread information
- Stack information
- Variable information
- Stepping through code
- Continue
- Multithreaded support

Some known limitations in this release include the following:

- Debug output is redirected to the serial console when debugging a DKM.
- The debugger will not function properly if any source files have spaces in their path.

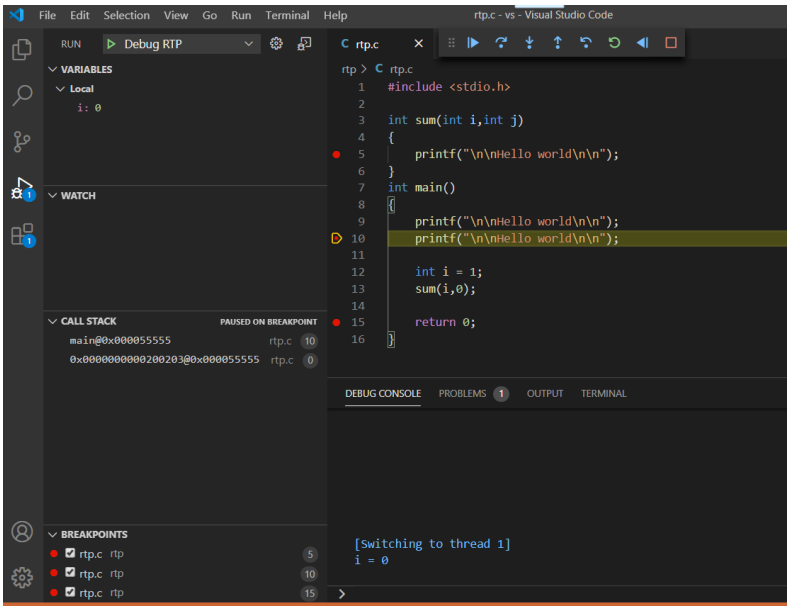
Procedure

1. Add one or more breakpoints in the source file.

   In the example shown in the next step, there is a breakpoint at line 10.

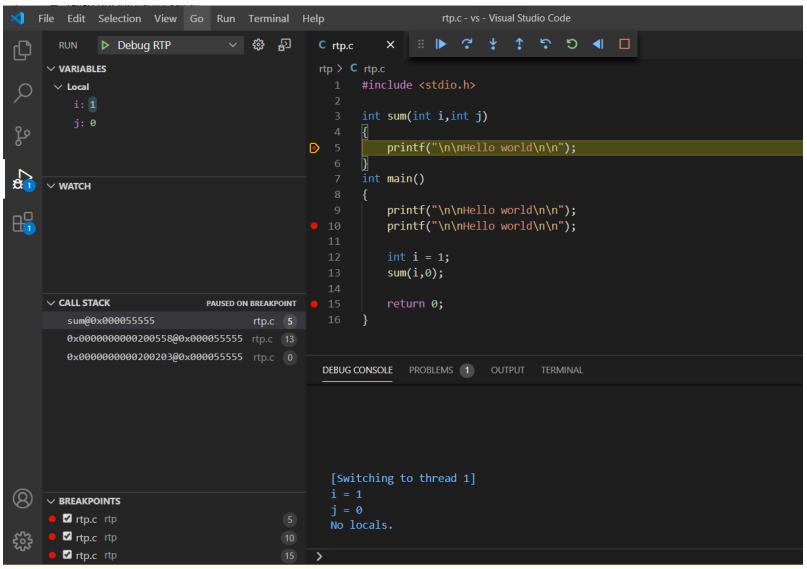
2. To run/debug an application, press the green side-arrow icon to start the debugging session.

   For example:



3. As required, add more breakpoints, and continue execution until the breakpoint is hit.

   For example, the following illustrates a new breakpoint at line 5 of the source file in the sample sum( ) function.



5. VSCODE DEBUG CONFIGURATION SETTINGS

These debug configurations are preset to enable execution of DKM applications or RTP applications from VSCode using the debugger.

Name (with Type)	Required or Optional (with default)	Description
cwd (string)	Required	Specifies the path to your project.
sdkPath (string)	Required	Specifies the local path to the SDK.
target (string)	Required	Specifies the path to your application executable.
targetType (string)	Required	Specifies the target type of the VxWorks executable (RTP or DKM). This is technically not required if **target** has a valid file extension (**.vxe** for RTPs and **.out** for DKMs).
arguments (string)	Optional	Specifies arguments to append after the executable. You can also use pipes.
autorun (string)	Optional	Specifies GDB commands to run when starting to debug.
debugger_args -debug (array)	Optional	Specifies additional arguments to pass to GDB. These are functionally executable arguments so the option and the value must be separate array entries; for example, ["-ip", "xxx.xxx.xxx.xxx"]. If the -debug option is specified, this enables debug output at the interpreter level.
dkmEntrypoint (string)	Optional (none)	Specifies the name of the DKM function that will be the debugging entry point. This is technically optional only for non-DKMs. It is required with DKMs.
env (object)	Optional	Specifies an environment overriding the gdb (and, in turn, also the process) environment.
gdbpath (object)	Optional (points to the Wind River wrdbg utility)	Specifies the path to the gdb executable or the command if accessible in the host's **PATH** environment variable. This should not be changed as any other GDB-equivalent utility will not be able to debug or run RTPs and DKMs in a VxWorks environment.
noDebug (boolean)	Optional (False)	Specifies if debugging features are enabled (True) or not (False). If False, you can only execute the application.
startGenericQemu (boolean)	Optional (False)	Specifies if the system automatically enables starting a generic QEMU image for each debug session.

Name (with Type)	Required or Optional (with default)	Description
stopQemu (boolean)	Optional (False)	Specifies if the QEMU instance should be terminated after debugging session has completed.
targetIp (string)	Optional (localhost)	Specifies the IP address of target board.
terminal (string)	Optional	If not set, this keeps program output in the VSCode console at the bottom. If set to the empty string on a Linux host, the program will spawn in a new console using **x-terminal-emulator**. Otherwise, on a Linux host, spawns the program with the specified terminal. If set to the empty string on a Windows host, spawns the program in a new console (no other console is supported).
valuesFormatting (string)	Optional	Specifies how to show variable values: **disabled: show value as is** **parseText: parse debuggers output text into structure** **prettyPrinters: enable debugger's custom pretty-printers, if there are any**
vxWorksKernelImage (string)	Optional (*sdkInstallDir* **/bsps/** *bspName* **/boot/vxWorks**)	Specifies the local path of the VxWorks kernel image.